education

Погружение в алгоритмы и структуры данных

Лекция

Илья Почуев
Павел Кулис
Константин Жуков
Максим Кулис

Илья Почуев

Старший backend-разработчик,
Вконтакте



education

Константин Жуков

Руководитель группы тестирования,
Вконтакте



education

Павел Кулис

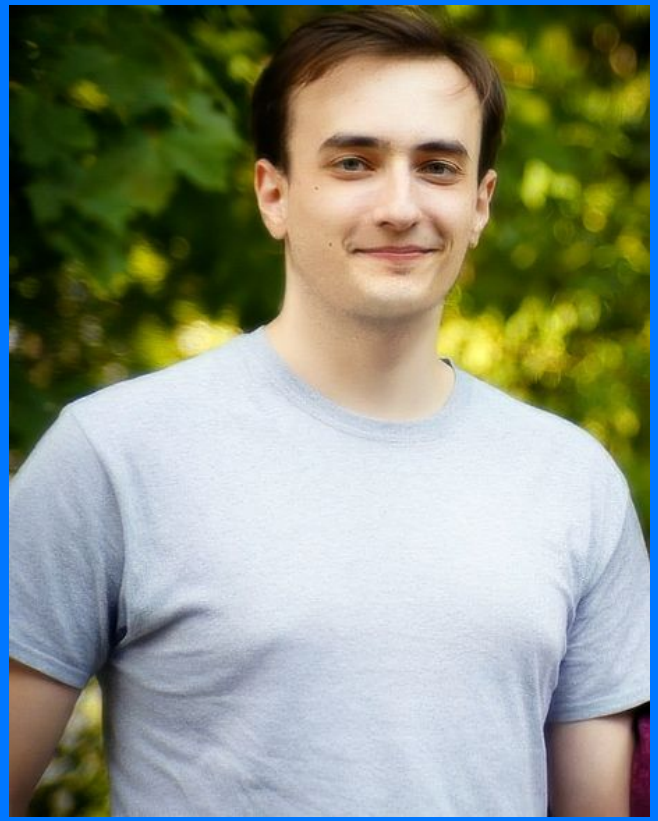
Старший разработчик, отдел разработки DWH



education

Максим Кулис

Старший разработчик, группа разработки inference платформы



education

Цель занятия:

Познакомить с курсом, его целями,
структурой, требованиями
и преподавателями

Немного организационных вопросов

- Обучение будет в формате лекция + семинар
- Лекция по вторникам, семинары по четвергам
- Во время лекции микрофон должен быть выключен
- Вопросы задаем в чат
- Актуальное расписание, ссылки на дз, описание занятий можно найти на VK Education
- Также на платформе нужно отмечать свое присутствие на занятии
- Обязательно заполнить профиль

О чем курс?

- Структуры данных
- Алгоритмы
- Подходы к решению задач
- Поговорим о прикладной пользе

Темы лекций

- Массивы
- Списки. Очереди и стек
- Хеш-таблица
- Алгоритмы поиск
- Сортировки
- Динамическое программирование
- Алгоритмы на строках
- Деревья и древовидные структуры
- Графы
- Префиксные суммы. Запросы на отрезках

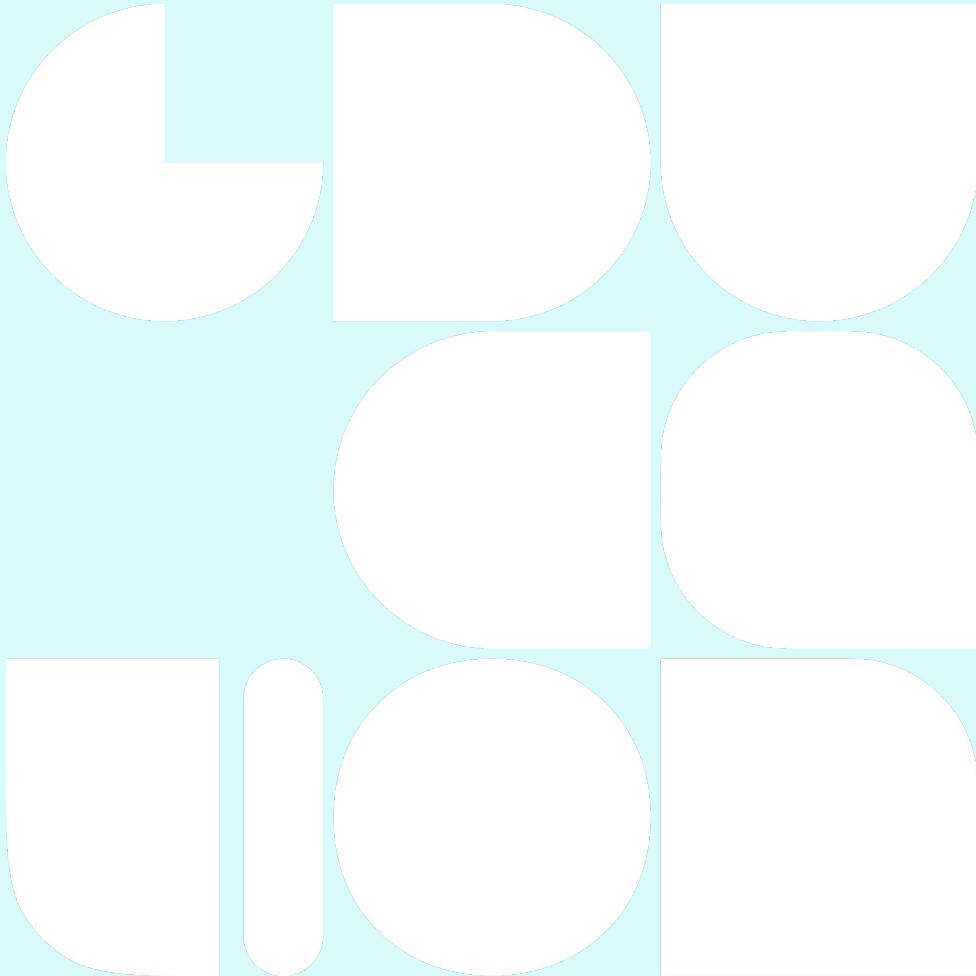
Чего не будет в курсе

- Языков программирования
- Математического обоснования

Зачем нам это?

- Хранение данных
- Обработка данных
- Понимание сложности и ресурсоемкости
- Часть профессиональной гигиены

Базовые структуры данных и оценка сложности алгоритма



Цель занятия:

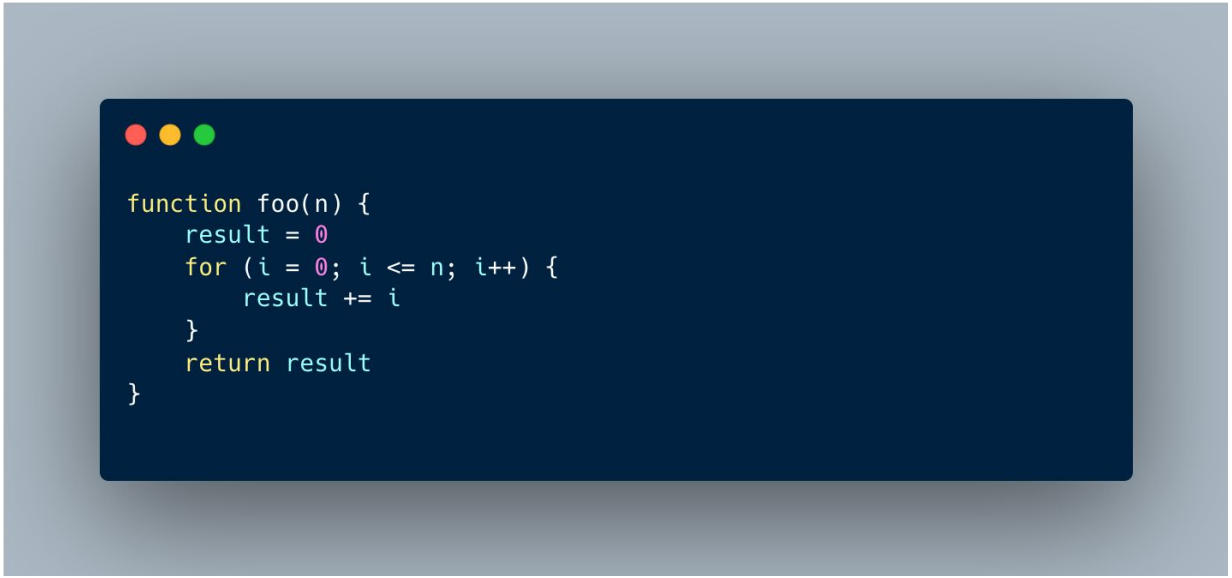
Познакомиться с основами оценки сложности алгоритмов с использованием нотации “О большое “ (Big O), изучить особенности линейных структур данных и понять концепцию амортизационной сложности.

Сложность алгоритма или Big O

- Как оценивать сложность?
- O(N) дает связь между входными значениями и итоговой оценкой сложности
- Виды сложности от O(1) до O(N!)
- Константы не влияют на сложность: O(2*N)~O(20*N)
- Учитываем все входные параметры
- Всегда “округляем” до наихудшей сложности
- Оцениваем сложность по потребляемой памяти



Какая сложность?



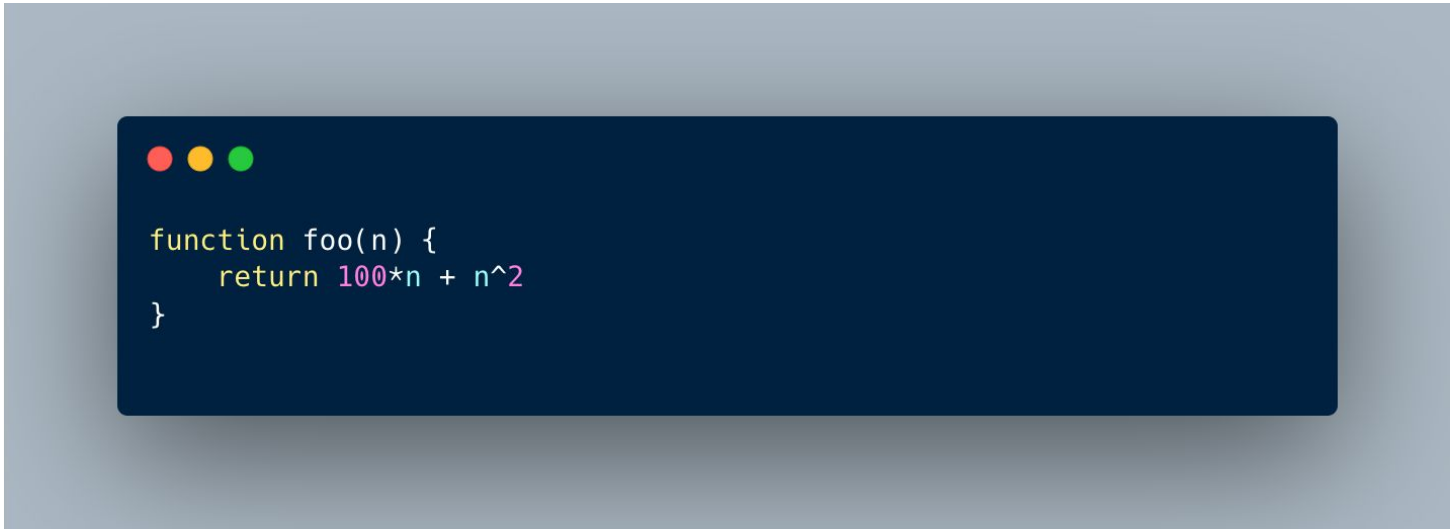
```
function foo(n) {
    result = 0
    for (i = 0; i <= n; i++) {
        result += i
    }
    return result
}
```

Какая сложность?

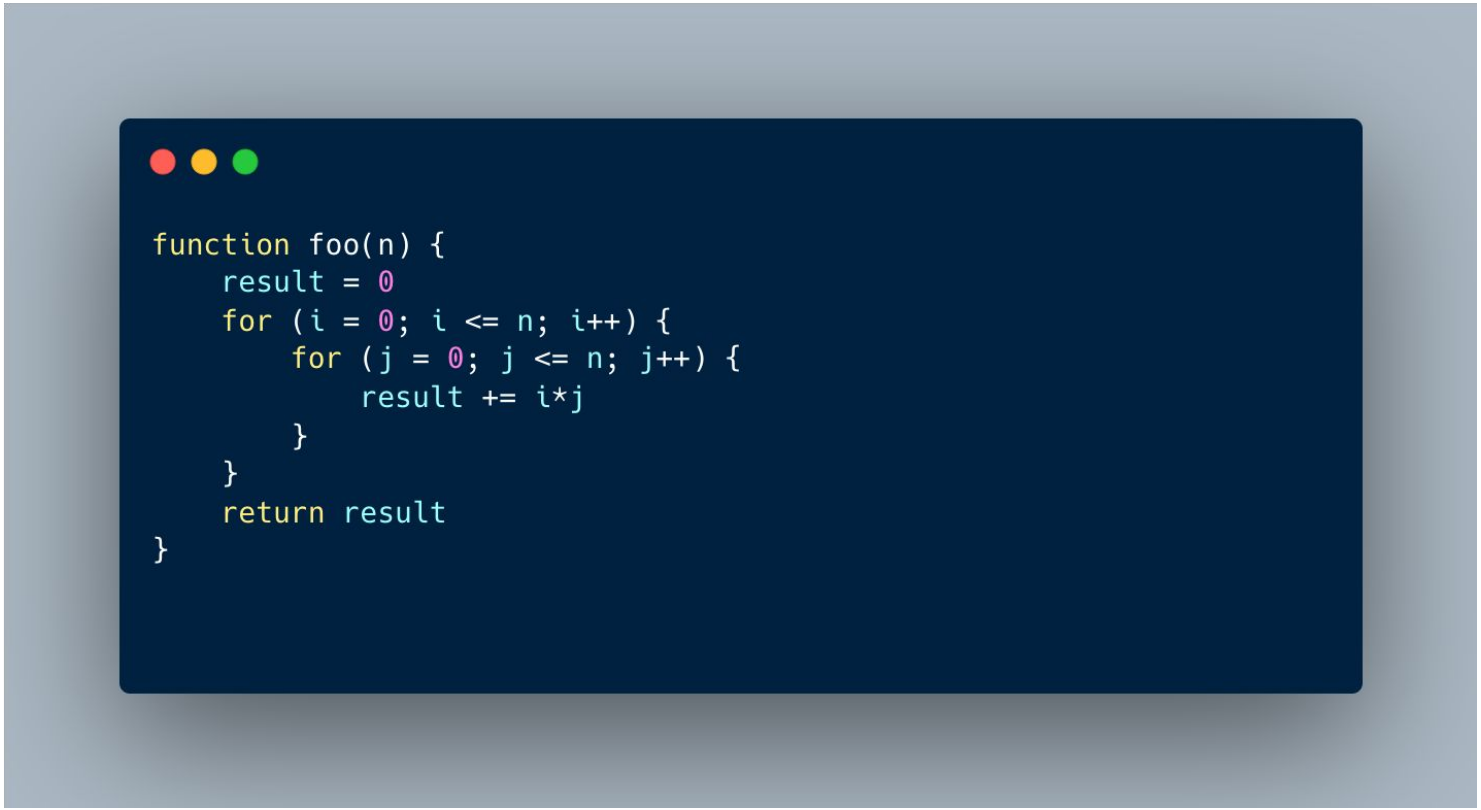


```
function foo(n) {
    result = 0
    for (i = 0; i <= n; i++) {
        result += i
    }
    for (i = 0; i <= n; i++) {
        result += i
    }
    for (i = 0; i <= n; i++) {
        result += i
    }
    return result
}
```

Какая сложность?



Какая сложность?



```
function foo(n) {
    result = 0
    for (i = 0; i <= n; i++) {
        for (j = 0; j <= n; j++) {
            result += i*j
        }
    }
    return result
}
```

Какая сложность?

```
function foo(n) {
    result = 0
    for (i = 0; i <= n; i++) {
        for (j = 0; j <= n; j++) {
            result += i*j
        }
    }
    for (i = 0; i <= n; i++) {
        result += i
    }

    return result
}
```

```
function foo(n, m) {
    result = 0
    for (i = 0; i <= n; i++) {
        for (j = 0; j <= m; j++) {
            result += i*j
        }
    }

    return result
}
```

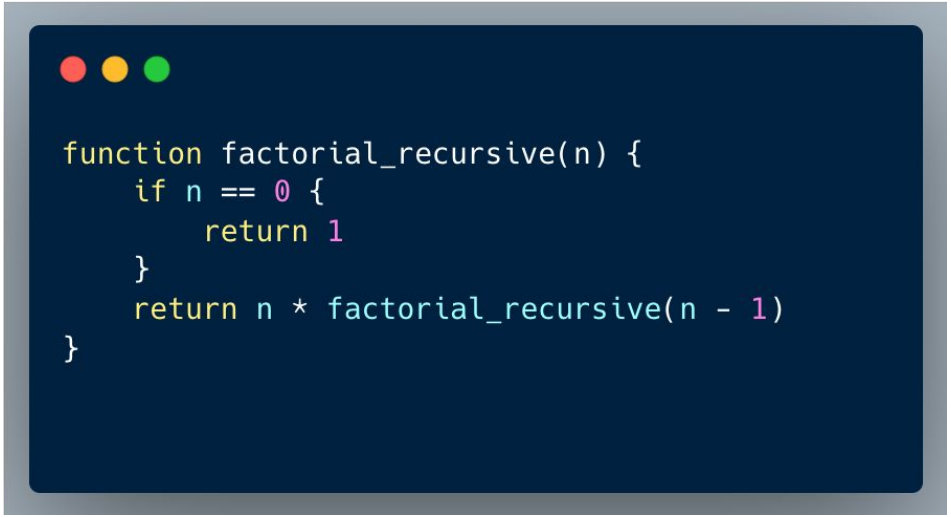
O(n!)



Вычисление факториала

Рекурсивный подход — факториальная сложность

n == 0 — базовый случай

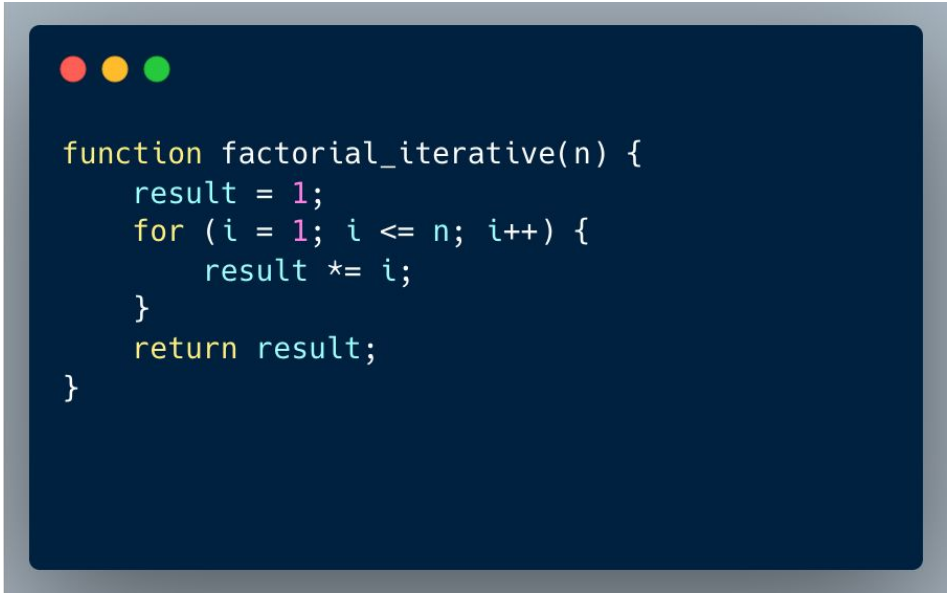


```
function factorial_recursive(n) {
    if n == 0 {
        return 1
    }
    return n * factorial_recursive(n - 1)
}
```

Вычисление факториала

Итерационный подход

Сложность O(n)



```
function factorial_iterative(n) {
    result = 1;
    for (i = 1; i <= n; i++) {
        result *= i;
    }
    return result;
}
```

Вычисление факториала

```
function factorial_recursive(n) {
    if n == 0 {
        return 1
    }
    return n * factorial_recursive(n - 1)
}

//Тоже O(n)
```

```
func factorial(n int) int {
    if n == 0 {
        return 1
    }
    return n * factorial(n-1)
}

func factorialIter(n int) int {
    result := 1
    for i := 1; i <= n; i++ {
        result *= i
    }
    return result
}

func BenchmarkFactorial(b *testing.B) {
    for i := 0; i < b.N; i++ {
        factorial(50)
    }
}

func BenchmarkFactorialIter(b *testing.B) {
    for i := 0; i < b.N; i++ {
        factorialIter(50)
    }
}
```

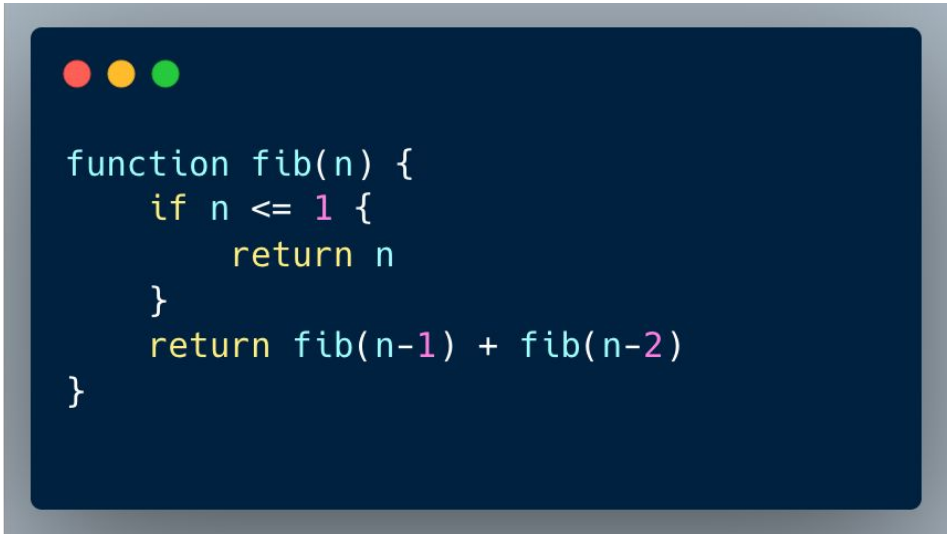
```
BenchmarkFactorial-12          16672078          68.24 ns/op          0 B/op          0 allocs/op
BenchmarkFactorialIter-12      75296479          14.75 ns/op          0 B/op          0 allocs/op
```

$O(2^n)$



Вычисление числа Фибоначчи

Рекурсивный подход



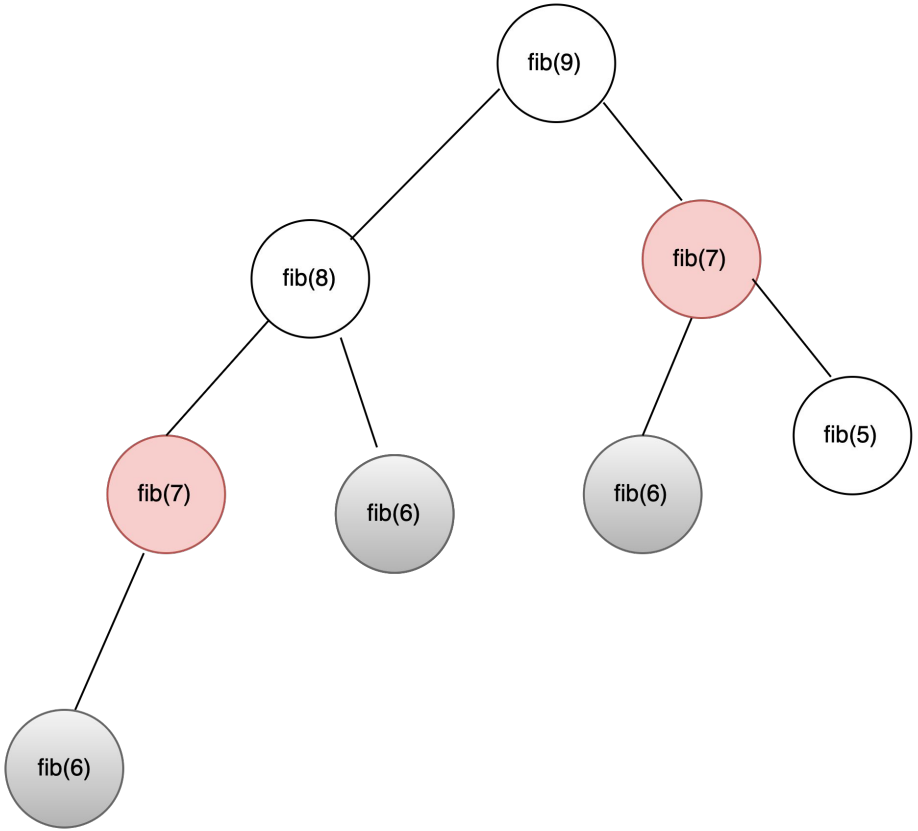
```
function fib(n) {
    if n <= 1 {
        return n
    }
    return fib(n-1) + fib(n-2)
}
```

Недостатки

- В коде, приведенном выше, есть два рекурсивных вызова

  return fib(n-1) + fib(n-2)

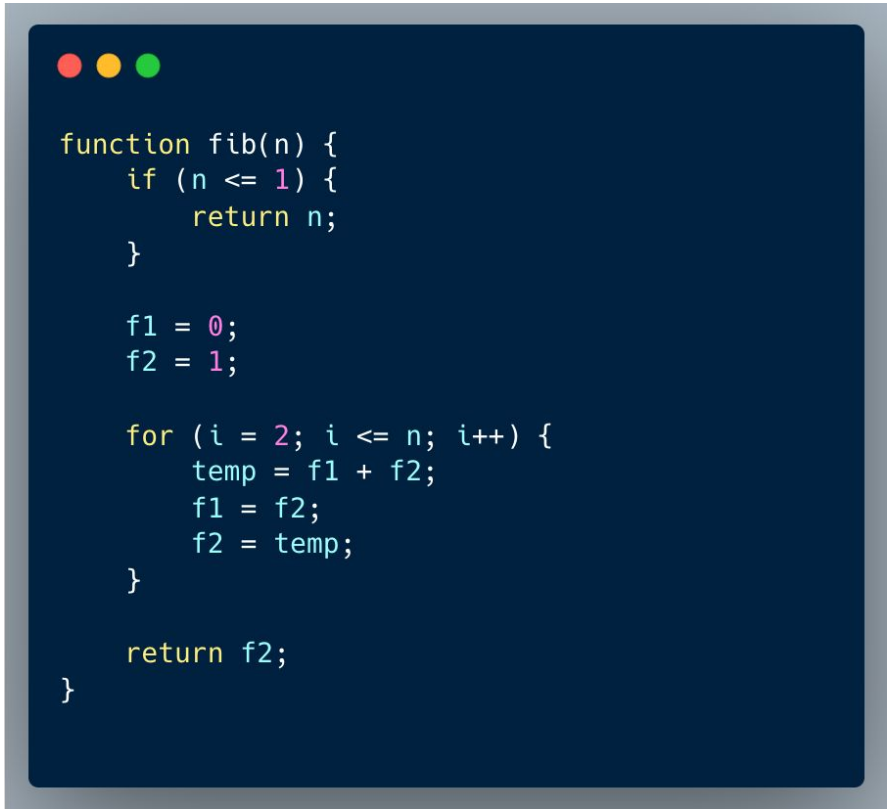
- Некоторые значения мы будем высчитывать по многу раз
- Каждый вызов fib ничего не знает о предыдущих вызовах
- На каждом вызове все значения вычисляются заново
- Несмотря на лаконичность рекурсивного подхода, происходят большие затраты на вычисления.



Вычисление числа Фибоначчи

Итерационный подход

Сложность O(n)



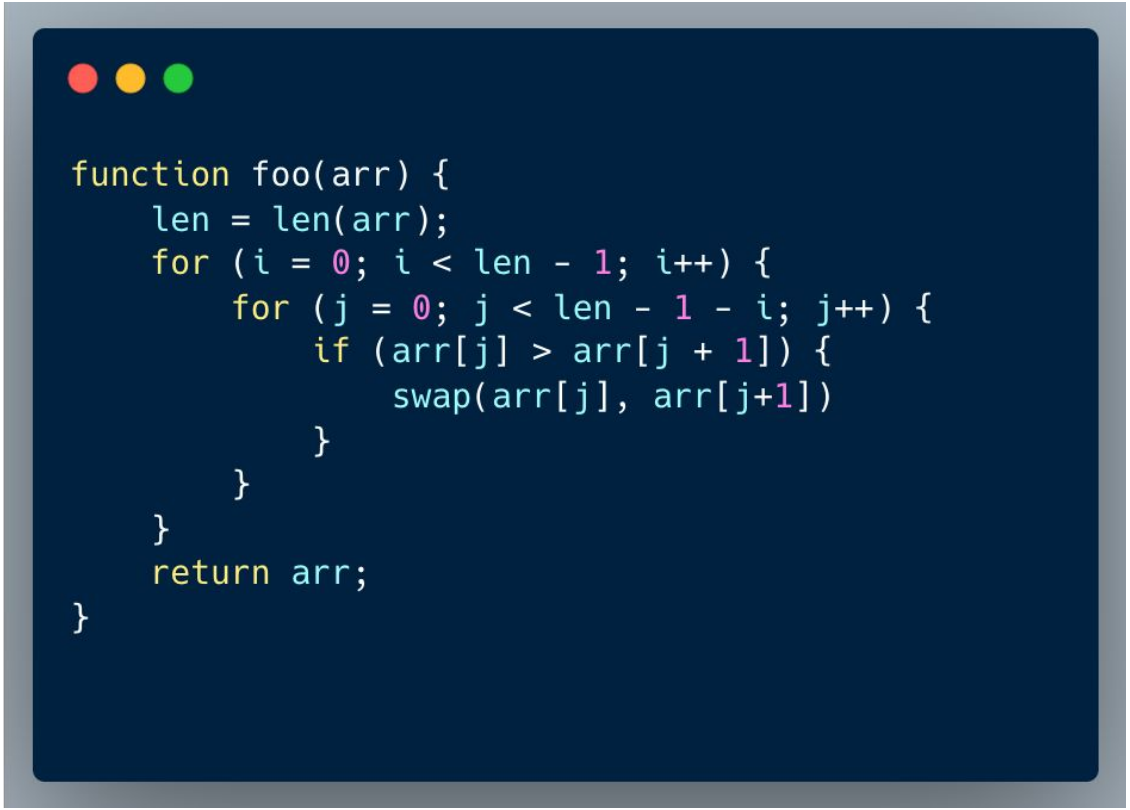
```
function fib(n) {
    if (n <= 1) {
        return n;
    }

    f1 = 0;
    f2 = 1;

    for (i = 2; i <= n; i++) {
        temp = f1 + f2;
        f1 = f2;
        f2 = temp;
    }

    return f2;
}
```

Квадратичная сложность



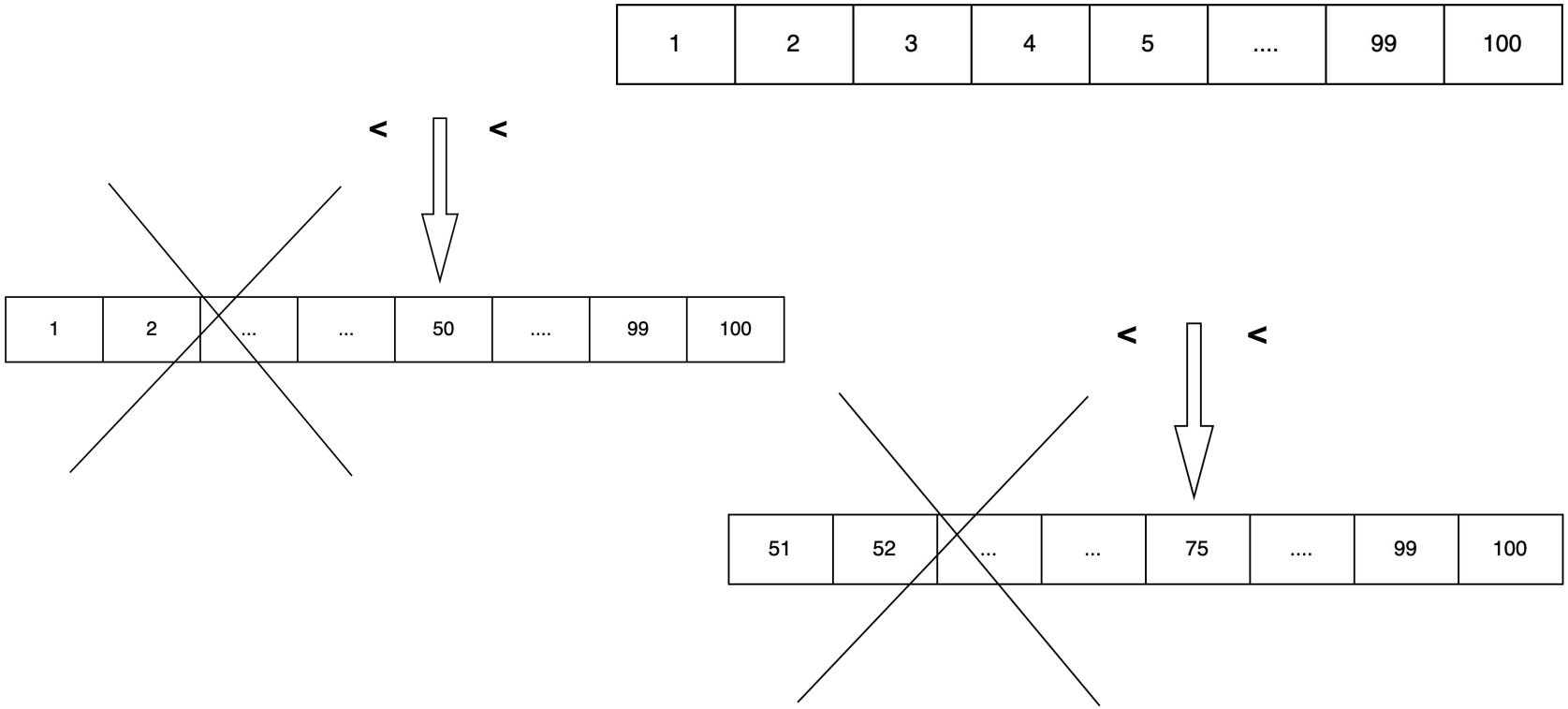
```
function foo(arr) {
    len = len(arr);
    for (i = 0; i < len - 1; i++) {
        for (j = 0; j < len - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j+1])
            }
        }
    }
    return arr;
}
```

Квадратичная сложность



```
function bubbleSort(arr) {
    bool sorted = false
    while !sorted {
        sorted = true
        for i = 0..len(arr)-1 {
            if (arr[i] > arr[i+1]) {
                swap(arr[i], arr[i+1])
                sorted = false
            }
        }
    }
}
```

Игра в угадай число



```
function binarySearch(data, l, r, target) {
    if (l > r) {
        return -1;
    }

    // Вычисляем индекс середины и округляем до целого числа в меньшую сторону
    middle = (l + r) / 2;

    if (data[middle] === target) {
        return middle;
    }

    if (data[middle] > target) {
        // Ищем в левой стороне
        // Правая граница смещается до middle - 1 включительно
        return binarySearch(data, l, middle - 1, target);
    } else {
        // Ищем в правой стороне
        // Левая граница смещается до middle + 1 включительно
        return binarySearch(data, middle + 1, r, target);
    }
}
```

O(logn)



Ω

- Нижняя граница сложности
- Тоже самое, что и O(n) только для самого лучшего случая
- Более быстрого значения, чем Ω для данного алгоритма быть не может

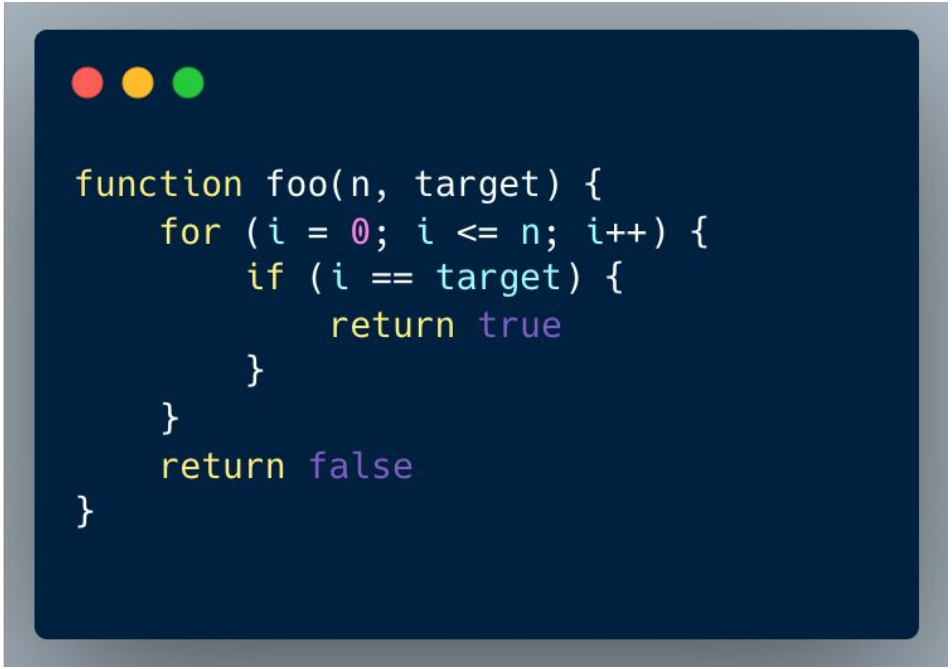


```
function bubbleSort(arr) {
   bool sorted = false
   while !sorted {
       sorted = true
       for i = 0..len(arr)-1 {
           if (arr[i] > arr[i+1]) {
               swap(arr[i], arr[i+1])
               sorted = false
           }
       }
   }
}
```

Θ

- Когда мы говорим о сложности алгоритма Theta, мы учитываем, что алгоритм будет работать где-то между двумя экстремальными случаями — наилучшим и наихудшим.

- Сложность алгоритма Theta представляет собой точную оценку, определяющую не только границы сверху и снизу но и худший, лучший и средний случай

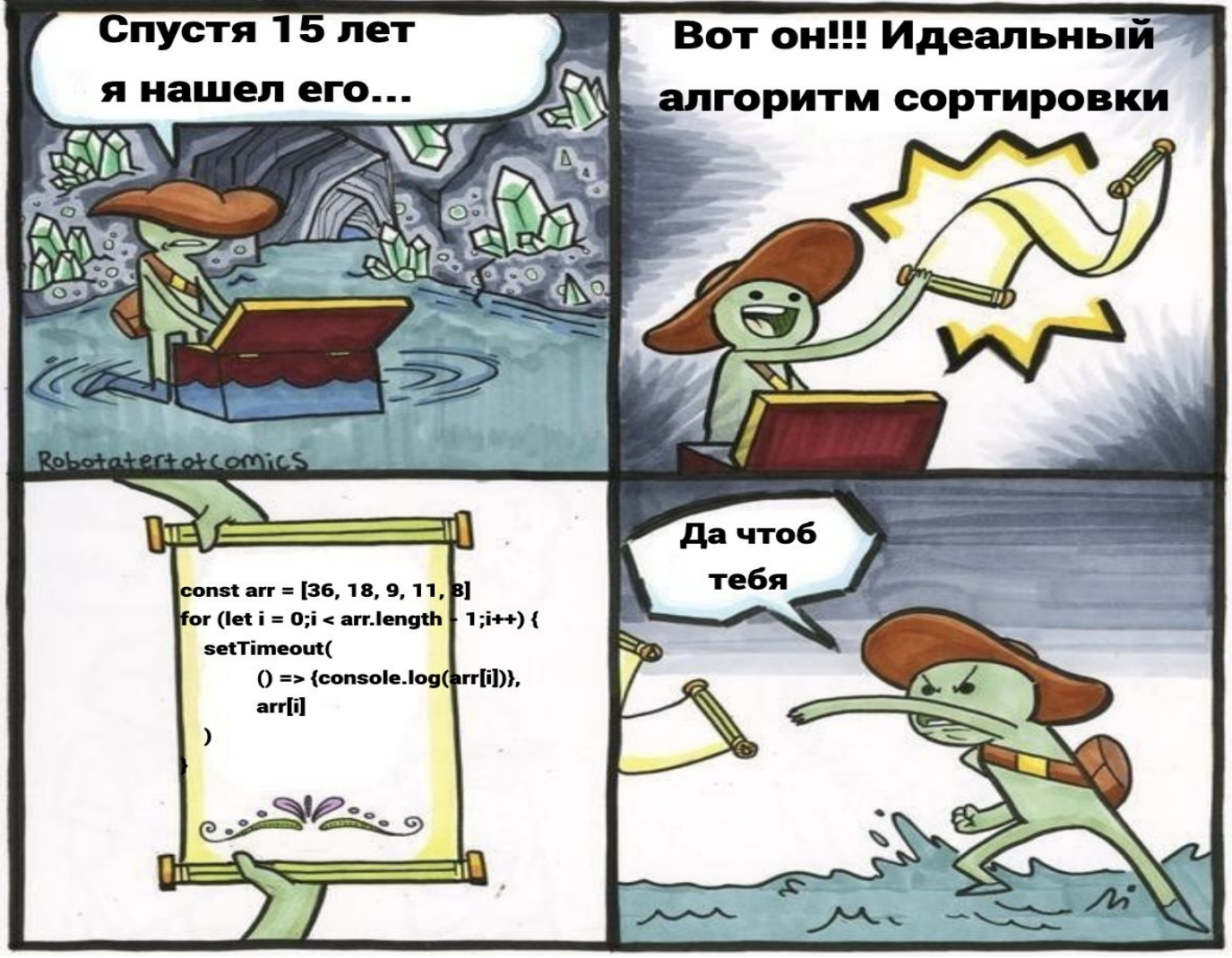


```
function foo(n, target) {
    for (i = 0; i <= n; i++) {
        if (i == target) {
            return true
        }
    }
    return false
}
```

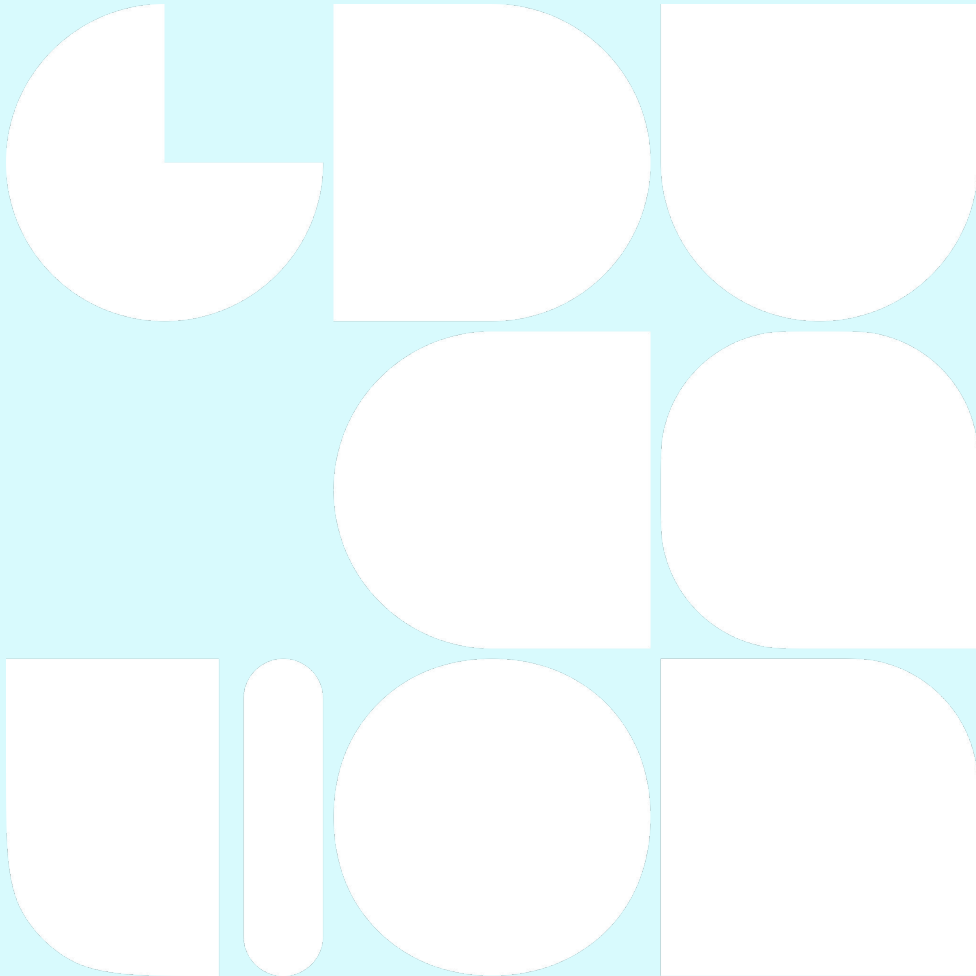
Еще один аспект

Алгоритмическая часть собеседований



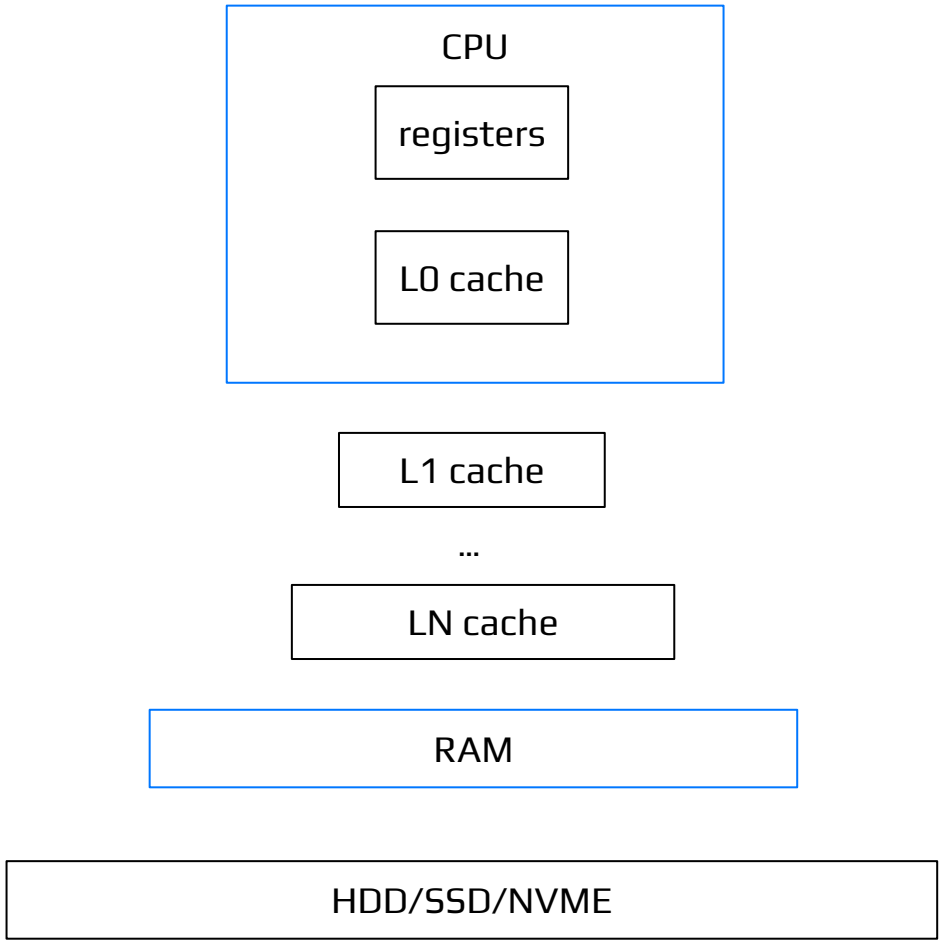

Спустя 15 лет
я нашел его...
RobotatertotComics
Вот он!!! Идеальный
алгоритм сортировки
const arr = [36, 18, 9, 11, 8]
for (let i = 0;i < arr.length - 1;i++) {
setTimeout(
() => {console.log(arr[i])},
arr[i]
)
}
Да чтоб
тебя

Ближе к железу

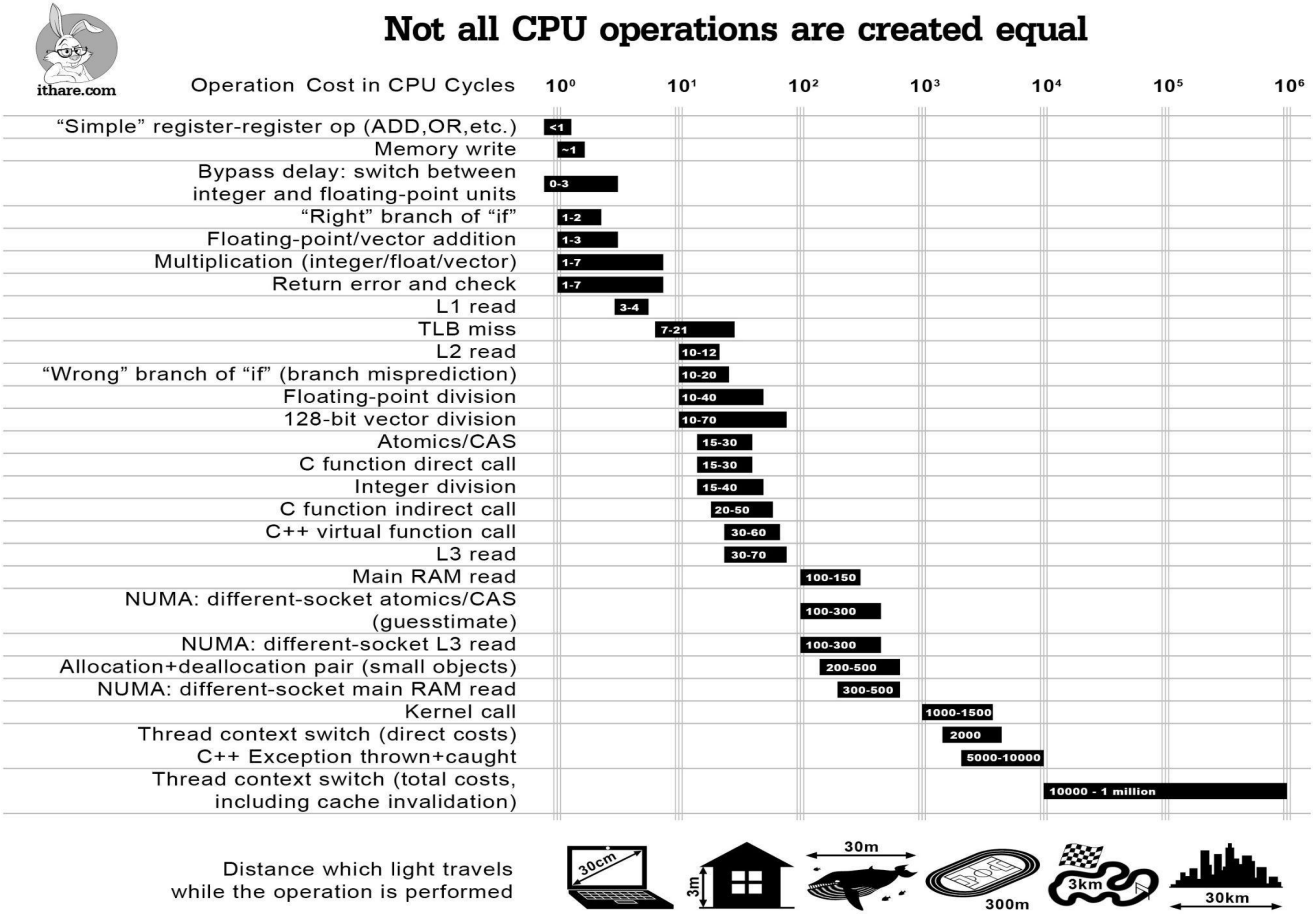


Архитектура компьютера





Цена одной операции



Массивы

- → Непрерывная область памяти заданного размера
- → Массив подразумевает под собой хранение однотипных данных, расположенных друг за другом в памяти
- → Доступ к элементу массива осуществляется посредством целочисленного индекса
- → Обращение к ячейке по индексу за константное время

Структура массива

- size количество элементов
- capacity объем массива
- type тип данных

```
Array struct {
    size     int
    capacity int
    type     int
}
```



Для массива из 8 элементов с типом int64 при условии, что на моей архитектуре 1 элемент с типом int занимает 8 байт, в памяти будет последовательно аллоцировано 64 байта



*Адрес=0x1000+0x10=0x1010

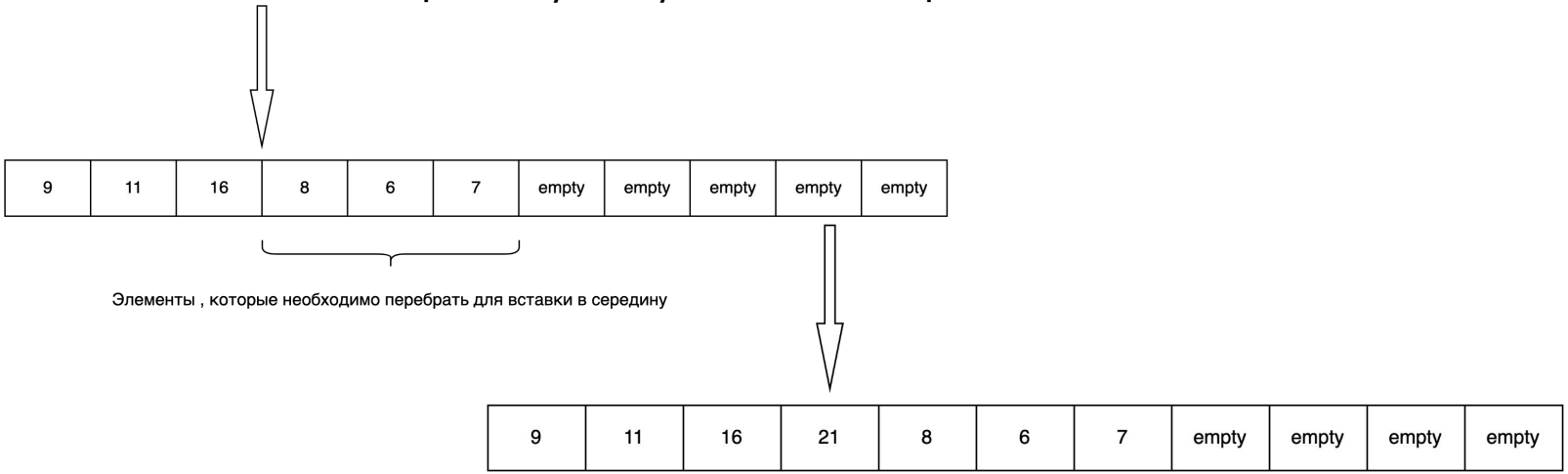
- Допустим нулевой элемент начался с ячейки памяти с номером 1000
- Тогда байты с 1000 по 1007 будут принадлежать первому элементу массива. Байты с 1008 второму и т.д.
- Зная, что все элементы располагаются последовательно легко получить доступ к каждому из них
- Элемент номер 4 можно найти по формуле: нулевой элемент плюс произведение индекса на размер каждой ячейки 1000 + 4*8 = 1032

Действия над массивом

- append() — вставка
- get(index) — получение элемента по индексу
- capacity — емкость массива, количество элементов которое он может в себя вместить
- size (size<=capacity) — количество заполненных ячеек массива

Вставка/удаление из середины массива

- Допустим мы хотим добавить новый элемент в середину массива
- Так как все элементы должны располагаться друг за другом, то нам необходимо последовательно переместить каждый элемент вперед
- По схожей логике реализуется удаление из середины массива



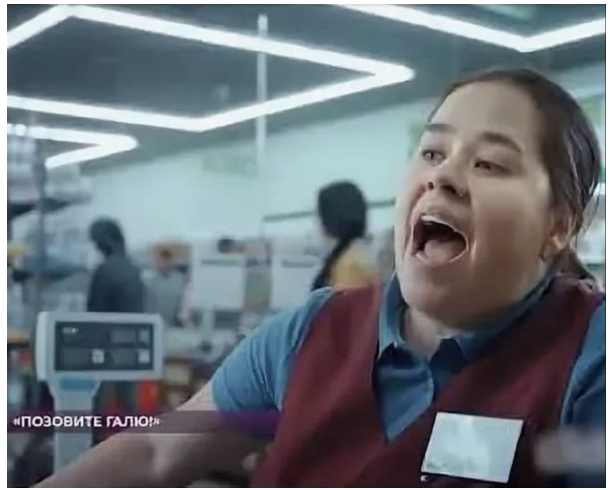
Сложность основных операций

- Получение данных O(1)
- Вставка в середину O(n)
- Вставка в конец O(1)
- Удаление O(n)

- Массив отлично подходит когда операций на получение элемента значительно больше чем вставок, идеально для readonly хранилища
- Мы заранее должны знать его размер
- Не подходит в ситуациях когда необходимо производить много вставок в середину

Пример: организация хранения товаров в памяти

- Один раз добавили все товары
- Чтобы узнать цену товара вы производите только выборку
- Операции вставки происходят гораздо реже



Саморасширяющийся массив

- Как быть если мы не всегда можем заранее знать размер массива?
- При этом есть необходимость получения элементов по индексу
- По-прежнему хотим O(1) при выборке

Как должна работать такая структура данных?

- Как я уже говорил у массива есть два основных параметра: size и capacity
- Если size становится равным capacity, то мы больше не можем вставлять в массив ничего
- Следовательно нам нужен алгоритм увеличения capacity
- Этот процесс называется реалокацией памяти
- Нам нужно заново найти свободное непрерывное пространство в памяти, чтобы мы могли перенести наши уже существующие элементы последовательно и плюс чтобы там было место под новый элемент
- Скопировать все значения из старого массива в новый
- Вставить новый элемент
- Удалить старый массив, чтобы избежать утечки по памяти
- Попробуем реализовать похожий на описанную выше процедуру в коде

Попробуем подобрать алгоритм увеличения capacity

- Сделаем capacity равным единицы при инициализации
- Попробуем понять, как лучше увеличивать емкость массива
- Попробуем увеличивать на 5 всякий раз когда мы пытаемся вставить элемент в заполненный массив

```python
import ctypes

class DynamicArray(object):
    def __init__(self):
        self.size = 0
        self.capacity = 1
        self.array = self.make_array(self.capacity)

    def cap(self):
        return self.capacity

    def append(self, element):
        if self.size == self.capacity:
            self.resize()
            print(f"slow: {self.cap()}")
        else:
            print('fast')

        self.array[self.size] = element
        self.size += 1

    def resize(self):
        # как правильно увеличивать capacity?
        new_cap = self.capacity + 5
        new_array = self.make_array(new_cap)

        for i in range(self.size):
            new_array[i] = self.array[i]

        self.array = new_array
        self.capacity = new_cap
```

```
fast
slow: 6
fast
fast
fast
fast
slow: 11
fast
fast
fast
fast
slow: 16
fast
fast
fast
fast
slow: 21
fast
fast
fast
fast
slow: 26
fast
fast
fast
fast
slow: 31
```

Сложность реалокации

- Должны различать две ситуации вставки в конец массива:
- Когда нам не надо увеличивать capacity — сложность O(1)
- Когда нужно увеличивать — мы копируем все значения в новый массив, а значит сложность стремится к O(n)
- Как тогда правильно подсчитать сложность, понимая что O(n) будет далеко не всегда?
- Проведем множество операций вставки, подсчитаем общее количество элементарных действий и время на их выполнение и разделим общую сложность на количество операций
- Такая усредненная сложность называется амортизированной сложностью, а анализ называется амортизационным.

- При заполнении массива 10000-ми элементов метод resize будет вызываться 2000 раз. Это не совсем то, что нам нужно
- При увеличении емкости на 10 элементов — 1000 реалокаций и т.д.
- Сама по себе идея увеличивать на константу не очень хороша, так как чем больше элементов в массиве тем больше будет реалокаций а амортизационная сложность будет стремится к O(n)
- Самым оптимальным будет увеличивать сложность в константное значение, например в 2.
- Количество вызовов функции resize будет всего 15
- Количество простых операций при увеличении capacity на 5 = 9997001
- Количество простых операций при увеличении capacity в 2 раза = 16384

```
2
4
8
16
32
64
128
256
512
1024
2048
4096
8192
16384
```

```python
def resize(self):
    new_cap = self.capacity + 5
    new_array = self.make_array(new_cap)

    for i in range(self.size):
        self.count += 1      # 9997001
        new_array[i] = self.array[i]

    self.array = new_array
    self.capacity = new_cap
```

```python
def resize(self):
    new_cap = self.capacity * 2
    new_array = self.make_array(new_cap)

    for i in range(self.size):
        self.count += 1      # 16384
        new_array[i] = self.array[i]

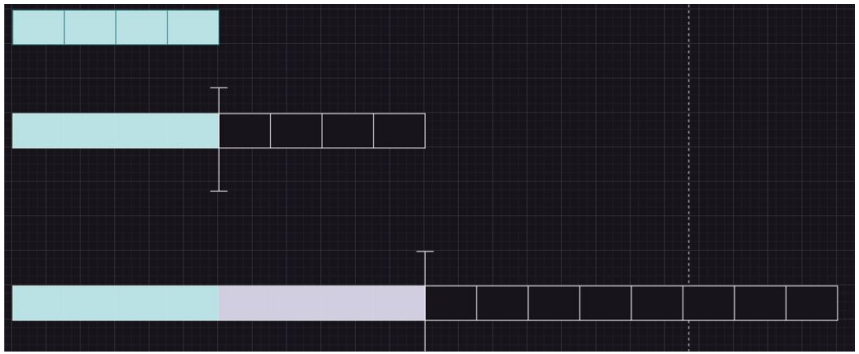
    self.array = new_array
    self.capacity = new_cap
```

Краткое резюме по амортизационной сложности реализации в два раза

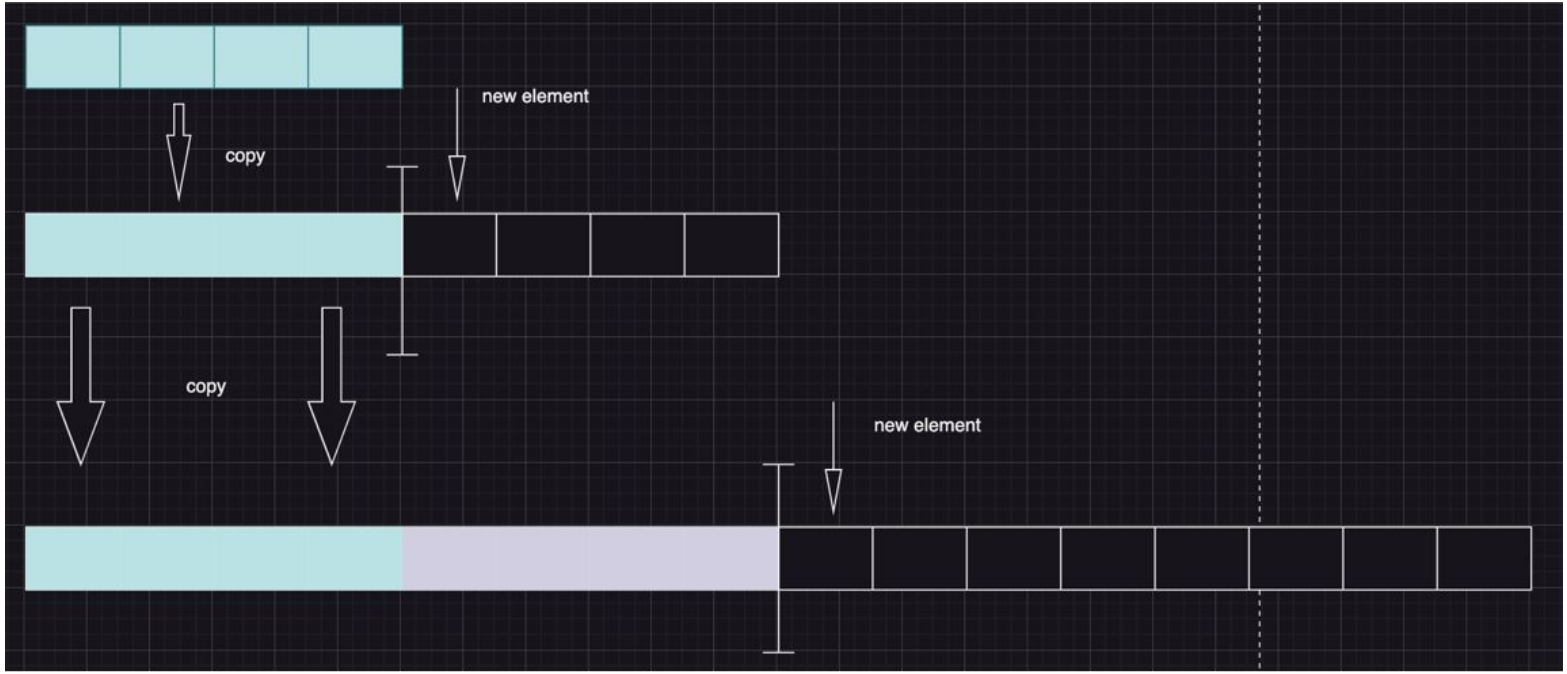
- До вызова метода resize, когда у нас есть место в массиве, мы произвели n вставок, каждая из которой занимала O(1)
- При реалокации мы произвели вставку, которая заняла O(n)
- В итоге у нас есть n + 1 операция, которые у нас заняли 2n (одно n до вызова resize, другое n непосредственно в момент вызова resize)
- Все это можно представить в виде 2n/(n + 1) что в нотации о большое эквивалентно O(1)

Итоги:

- ➔ Не обязательно знать конечный размер массива
- ➔ При превышении capacity создается новый массив с capacity*2
  (в какое-то константное значение)



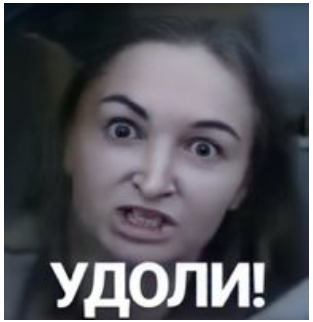
Аллокация памяти происходит каждый раз, когда мы пытаемся Вставить элемент в массив, в котором size == capacity



Помимо аллокации памяти необходимо скопировать все элементы из предыдущего массива
вставка в таком случае O(n)

А как удалять?

- Первое что приходит на ум — раз мы при добавлении увеличиваем емкость в два раза, то возможно при удалении надо следовать той же стратегии и уменьшать capacity в два раза когда массив освободился наполовину?
- Тут надо быть аккуратным, сейчас мы почти попали в ловушку. Дело в том, что если после уменьшения capacity в два раза, сразу последует вставка, то нам придется вновь аллоцировать в два раза больше памяти. А если эти операции будут повторяться? То мы рискуем получить в своем алгоритме сложность O(n)
- Уменьшают в два раз объём, когда справедливо равенство size = capacity/4. То есть когда реальное количество элементов в массиве в 4 раза меньше чем его ёмкость, тогда уменьшают capacity в 2 раза





Спасибо за внимание!

И хорошего вечера:))

education